Probabilistic Revenue & ROAS Forecasti for E-commerce Marketing

An AI-Assisted Forecasting Utility for Digital Marketing Agencies

AIgnition 2026 | NetElixir Hackathon | Technical Project Report

Team	Sigmoid
Member	Rehaan Ahmad Khan
College	JSS University, Noida
Repository	github.com/RAK2315/netElixir
Report date	19 July 2026

7.8%	89-94%	30/60/90d	30/30
blended revenue WAPE	interval coverage	forecast windows	edge cases passed

This report documents the full solution end to end: the problem, the data, the two-layer architecture, the forecasting model, the AI insight layer, testing, and how to reproduce it.

1. Executive Summary

This project is an AI-assisted forecasting utility that predicts e-commerce revenue and ROAS (Return on Ad Spend) as probabilistic ranges over 30, 60 and 90-day windows, across the Google, Microsoft/Bing and Meta advertising channels, and explains each forecast with a Large Language Model (LLM) causal-inference layer.

The submission is judged in two very different ways, and the entire design flows from that:

- An automated pipeline clones the repository, installs pinned dependencies, drops in held-out test data, runs a single command, and scores the resulting predictions file. It runs fully offline from a pre-trained, committed model. Any failure scores zero.
- Human judges assess technical soundness, AI integration, product thinking, and engineering quality.

To satisfy both, the codebase is split into two layers: a deterministic, offline Scoring Core that guarantees a valid predictions file, and a Product/Demo layer (an interactive dashboard and the LLM insights) that never sits on the scored path. The forecasting model is a conformalized quantile-regression gradient-boosting ensemble that achieves 7.8% aggregate revenue error with honestly calibrated uncertainty bands.

2. The Problem

Digital marketing agencies must estimate future business outcomes before budgets are deployed. This is hard: performance is shaped by seasonality, shifting user behaviour, inconsistent campaign structures, varying channel efficiency, and incomplete analytics. Existing workflows are spreadsheet-driven and disconnected across platforms.

The challenge asks for a practical forecasting utility that:

- forecasts aggregate e-commerce revenue and blended ROAS as probabilistic ranges, not single numbers;
- breaks forecasts down to channel, campaign-type and campaign level;
- supports 30/60/90-day planning windows and 'what-if' media-budget simulation;
- treats existing channel attribution as the source of truth (no custom attribution or full MMM); and
- adds an LLM layer for forecast explanation, anomaly interpretation and operational risk flags.

3. The Dataset

Three heterogeneous, daily, campaign-level feeds were provided. Each has a different schema, and one has a subtle data trap:

Google Ads — 19,272 rows, 92 campaigns

cost in micro-units (divide by 1e6); revenue = conversions value

Microsoft / Bing — 2,873 rows, 28 campaigns

native currency revenue and spend

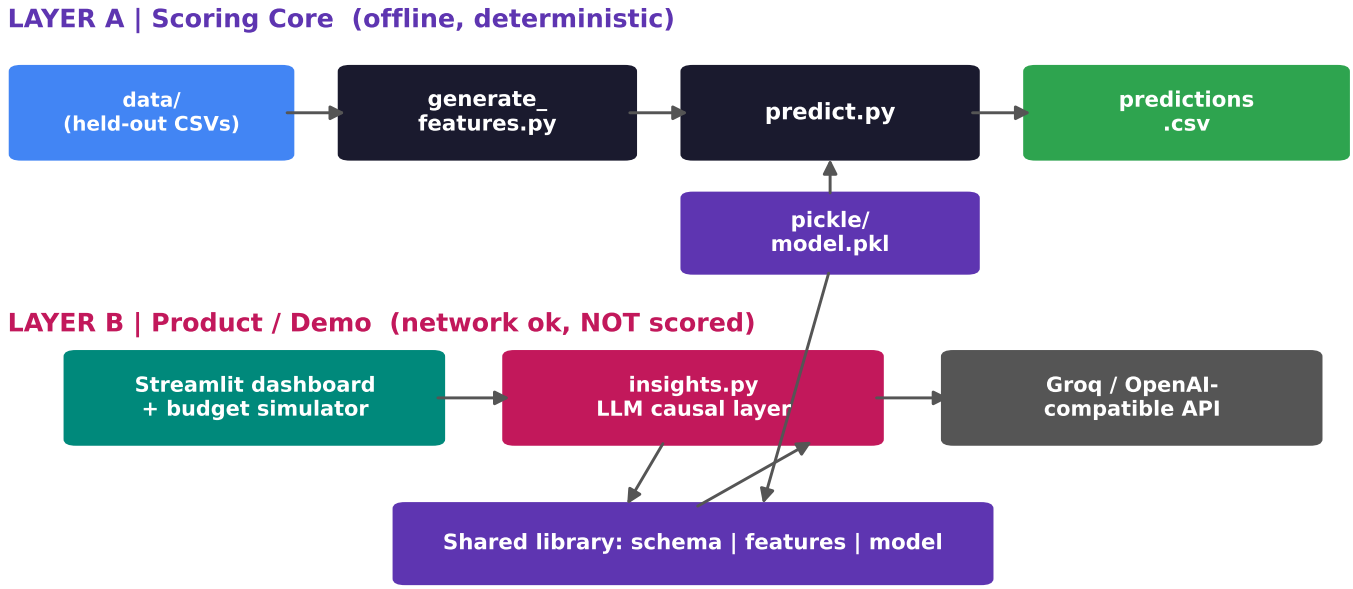
Meta Ads — 3,417 rows, 16 campaigns

NO revenue column; 'conversion' is a broad metric (~1.66M), not purchases

Data spans roughly January 2024 to June 2026 — enough history for seasonal, aggregate-period forecasting.

4. Solution Architecture

The system is split into two layers so neither compromises the other. The Scoring Core is deterministic and offline; the Product layer may use the network but is never invoked by the scored command.



Because the LLM and dashboard never run inside run.sh, the scored command can never fail for a network or API-key reason — a common way competing submissions score zero.

5. Data Unification & the Meta Revenue Problem

A single module (schema.py) auto-detects each file by its column signature (not filename, so the graders' renamed files still load), normalises units and campaign-type names, de-duplicates, and coerces messy values — producing one unified long table the rest of the system consumes.

The Meta revenue trap

Meta reports no revenue, and its 'conversion' field sums to ~1.66 million— clearly not purchases (Google shows ~$88 average order value). Multiplying that count by an AOV would fabricate ~$146M of revenue. Instead we impute Meta revenue as spend × assumed ROAS, where the assumed ROAS is the historical blended ROAS of the channels that DO report revenue (Google + Bing, ~4.75). This keeps Meta proportionate to spend and is flagged everywhere as a modelled, not measured, figure. Catching this is a key differentiator; many teams will not.

6. Feature Engineering

Forecasting is framed as tabular, direct-multi-horizon supervised learning. One training example asks: standing at cut-off date t, given a campaign's recent history and a planned spend for the next H days, what revenue will it earn in the window (t, t+H]?

- No leakage — every feature uses data on or before t; the label is strictly the future window.
- Horizon (30/60/90) is a feature, so a single model serves all windows.
- Planned spend is a feature — so budget-response forecasting is native (change spend, get new revenue).
- Features include trailing spend/revenue/conversions over 7/14/30/60/90 days, trailing ROAS, activity and momentum signals, revenue volatility, run-rate, and seasonality of the forecast window.

7. The Forecasting Model

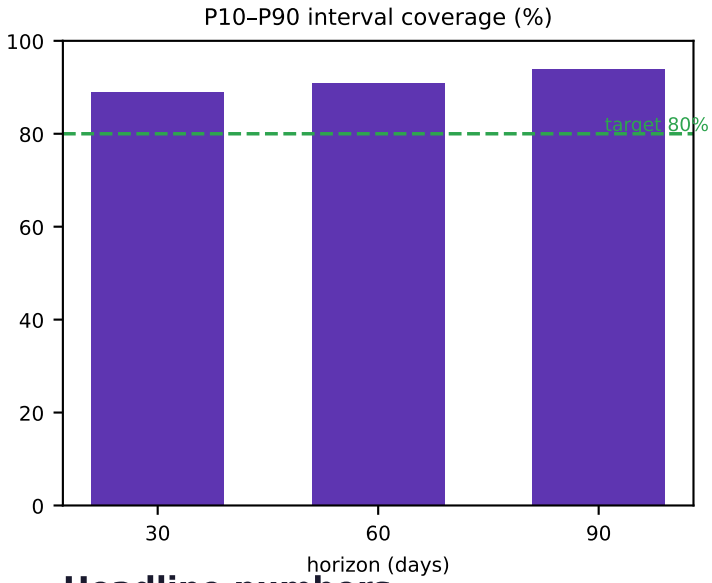
Three LightGBM gradient-boosted quantile regressors predict the 10th, 50th and 90th percentiles of window revenue. Gradient boosting handles the mixed, non-linear, heterogeneous feature set well, is fast and offline at inference, and (using the native Booster API) pickles with zero scikit-learn dependency — avoiding the single most common cause of submission failure.

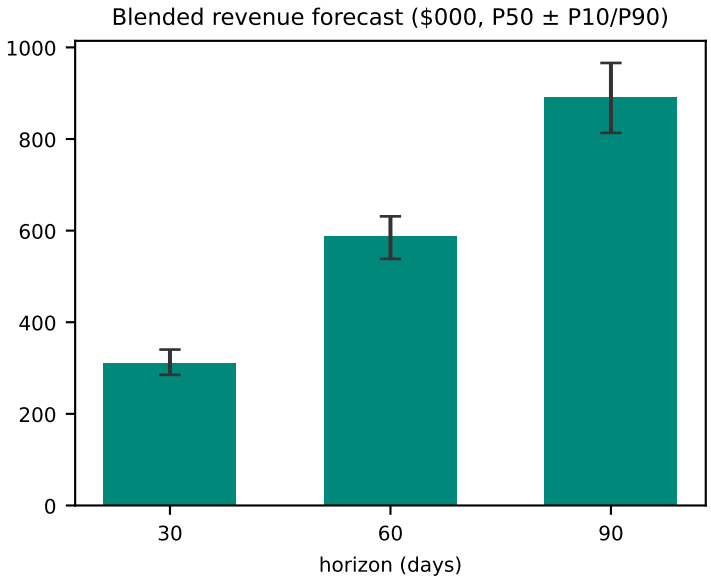
Conformalized Quantile Regression (honest uncertainty)

Raw quantile models are often mis-calibrated. Conformalized Quantile Regression (CQR) measures, on a held-out calibration slice, how far outcomes fall outside the predicted band, then widens the band per horizon to reach the target coverage — a distribution-free guarantee. Campaign-level forecasts aggregate to channel / type / blended by Monte-Carlo, so aggregate ranges are coherent rather than a naive sum of quantiles. ROAS = revenue / planned spend at each level.

8. Backtest Results

Evaluation uses a walk-forward split (65% train, 15% conformal calibration, 20% most-recent test). Aggregate error is low because campaign-level noise diversifies away when summed— the level at which budgets are actually set.





Headline numbers

- Blended-grain revenue error (P50 WAPE): 7.8% — the number agencies budget on.
- Campaign-grain revenue error (P50 WAPE): 30.7% (expected for noisy per-campaign forecasts).
- Interval coverage at 30 / 60 / 90 days: 89% / 91% / 94% against an 80% target (honest, slightly conservative).

9. Output Format

Per the organizers' guidance, the scored file mirrors the input/training columns plus the forecasted metrics. One row per campaign per horizon:

```
channel, campaign_id, campaign_name, campaign_type, horizon_days,
Revenue, ROAS, Revenue_p10, Revenue_p90, ROAS_p10, ROAS_p90
```

Revenue and ROAS are the P50 point forecasts; the _p10/_p90 columns carry the probabilistic range the brief requires. The file is written fresh on every run.

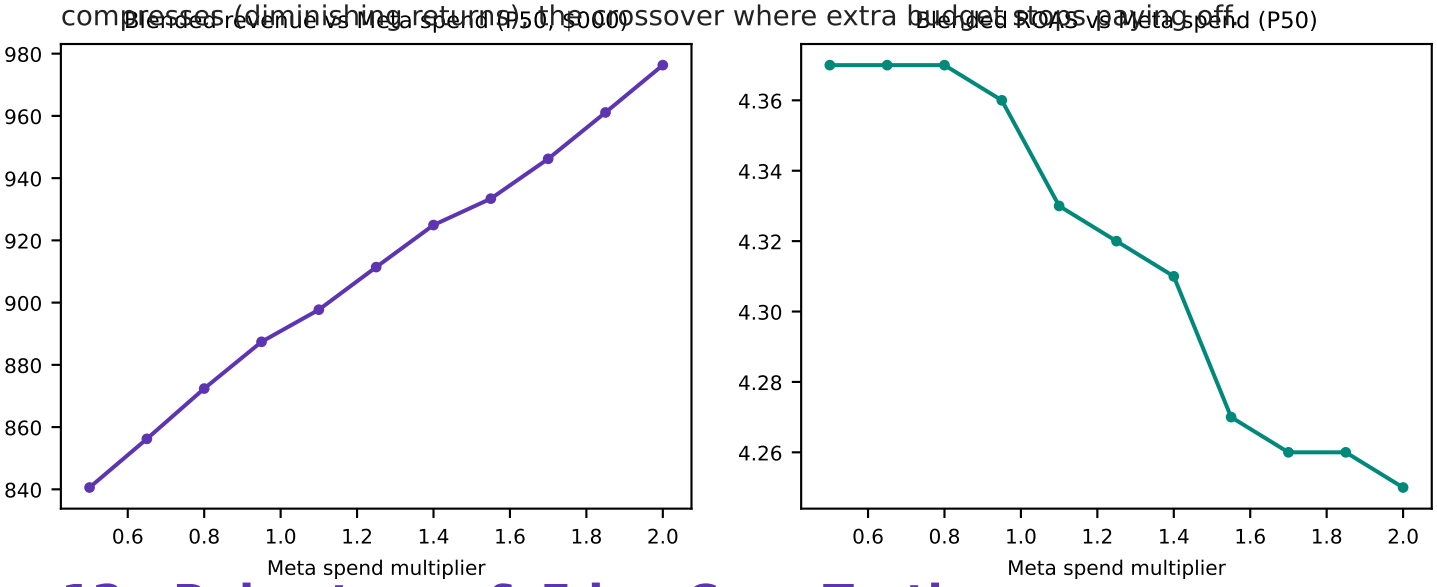
10. AI Causal-Insight Layer

The LLM turns the numeric forecast into an analyst-style briefing with four sections: Forecast Summary, Causal Drivers, Anomalies & Risks, and Recommended Actions. Two design choices matter:

- Provider-agnostic — it speaks the OpenAI-compatible protocol over plain HTTP, so Groq (default, free, fast), OpenRouter, xAI Grok or OpenAI all work via environment variables.
- Grounded and safe — the model only interprets numbers we compute (it never invents figures), and a deterministic rule-based fallback renders the same briefing offline, so the demo always works and the scored pipeline has zero LLM dependency.

11. Demo Application

A Streamlit dashboard ties the story together: probabilistic KPI cards, a revenue trajectory with a widening forecast cone, channel and campaign-type breakdowns, a live budget simulator, the AI insight panel, and a data-quality report that surfaces the Meta imputation. Below: the budget response curve for Meta at the 90-day horizon — revenue rises with spend while ROAS compresses (diminishing returns), the crossover where extra budget stops paying off.



12. Robustness & Edge-Case Testing

Because the graders replace the data with held-out rows, the loader is hardened against messy input. A fuzz harness of 30 deliberately unthinkable scenarios was run through the full scored path; all 30 produce a valid (or valid-empty) output with no crashes.

Scenarios include: empty files, single rows, negatives, 1e18 and 1e-9 values, NaN metrics, '-'/'N/A'/'1,234.5' junk in numeric columns, mixed / timezone / invalid dates, whitespace in headers, unicode and emoji names, shuffled column order, a missing channel, an unseen campaign type, dormant campaigns, duplicate IDs across channels, and a 3,000-row single campaign.

Values that cannot yield a forecast (all-zero or negative spend) correctly produce a valid empty file rather than crashing — a crash would score zero.

13. Reproducibility & Submission Compliance

- Public GitHub repo; run.sh at root, executable (mode 100755), one command end-to-end.
- run.sh takes DATA_DIR, MODEL_PATH, OUTPUT_PATH with sensible defaults; data read dynamically.
- Trained model committed under pickle/ (4.1 MB, plain git — not LFS); loads under pinned versions.
- requirements.txt fully pinned; scored path needs only pandas, numpy, lightgbm, pyarrow.
- Deterministic (seeded); no absolute paths; no network at run time.

- Verified on a fresh clone in a clean virtual environment with held-out data.

14. How to Run

Scored pipeline (what the graders run)

```
pip install -r requirements.txt
./run.sh ./data ./pickle/model.pkl ./output/predictions.csv
```

Demo application

```
pip install -r requirements.txt -r requirements-app.txt
cp .env.example .env   # optional Groq key
streamlit run app/streamlit_app.py
```

Retrain (optional; the pickle is already committed)

```
PYTHONPATH=. python src/train.py
```

15. Repository Structure, Limitations & Future Work

```
run.sh                      requirements.txt / -app.txt
data/  pickle/model.pkl     src/common/schema.py
src/forecasting/{features,model,conformal}.py
src/{generate_features,predict,train,insights}.py
app/streamlit_app.py        tests/test_pipeline.py   docs/
```

Limitations

- Meta revenue is imputed (spend × assumed ROAS) — modelled, not measured.
- Baseline assumes spend continues at the trailing run-rate; the simulator models deliberate changes.
- Attribution is used as-is (no cross-channel de-duplication or full MMM, per the brief).

Future work

- Replace the Meta imputation with true Shopify / GA4 revenue.
- Add promotions / holiday calendars and richer seasonality; per-campaign conformal calibration.

Conclusion — the solution combines statistical rigour (calibrated probabilistic forecasts), operational realism (handling three messy channel schemas and the Meta data gap), and clear product thinking (budget simulation and AI-written insight), packaged to pass a strict offline scoring contract reliably.